ΕΡΓΑΣΤΗΡΙΟ ΛΕΙΤΟΥΡΓΙΚΩΝ ΣΥΣΤΗΜΑΤΩΝ · 7ο ΕΞΑΜΗΝΟ

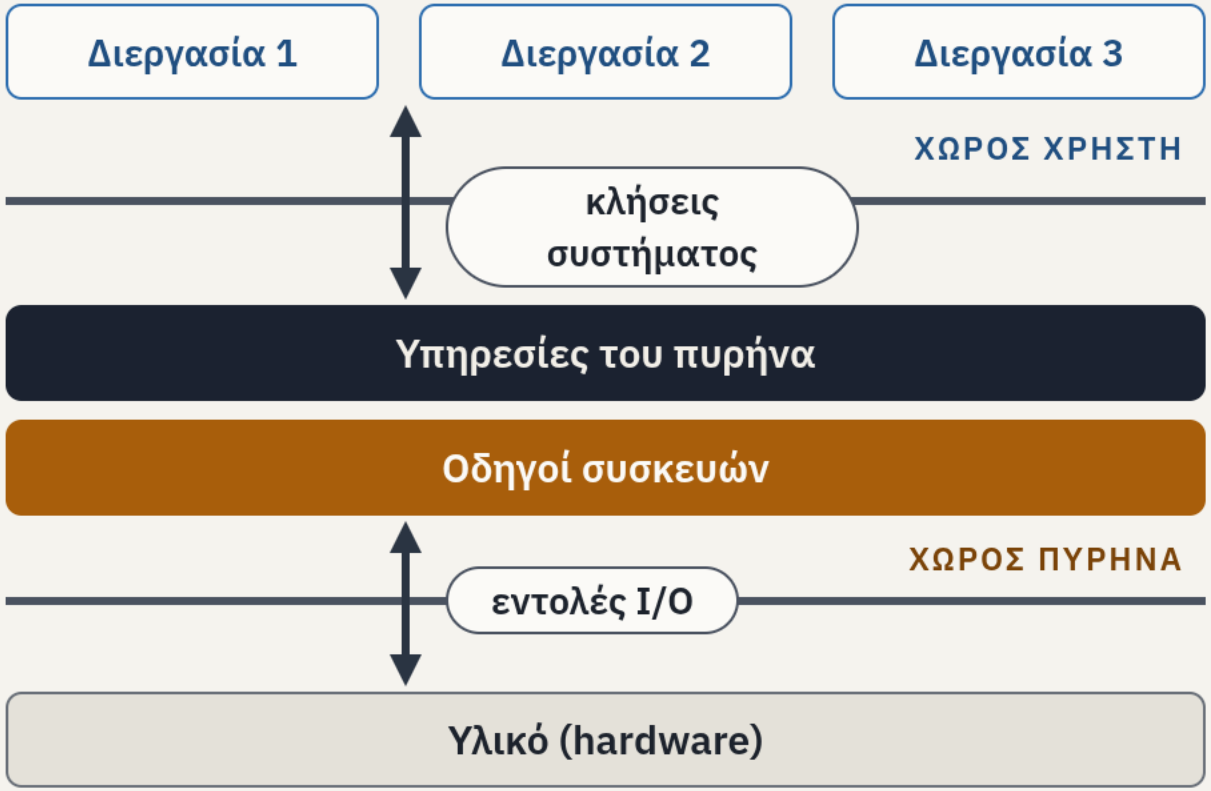
Από το user space στον πυρήνα

Εισαγωγή στον προγραμματισμό του πυρήνα του Linux

ΣΗΜΜΥ · ΕΜΠ

ΕΙΣΑΓΩΓΗ

Οργάνωση ενός σύγχρονου ΛΣ



Χώρος χρήστη

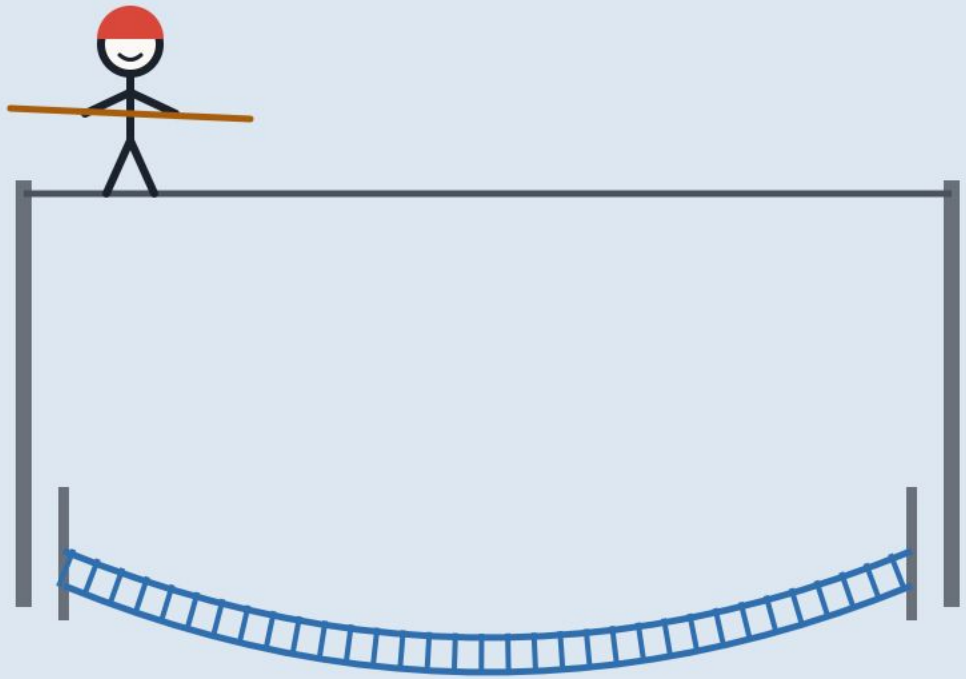
- απομονωμένη εκτέλεση
- πρόσβαση σε συγκεκριμένα κομμάτια μνήμης
- πρόσβαση στον πυρήνα μέσω κλήσεων συστήματος

Χώρος πυρήνα

- πρόσβαση παντού στη μνήμη
- ελεύθερη επικοινωνία με το υλικό (εντολές I/O)

User space

ένα bug → πέφτεις στο δίχτυ



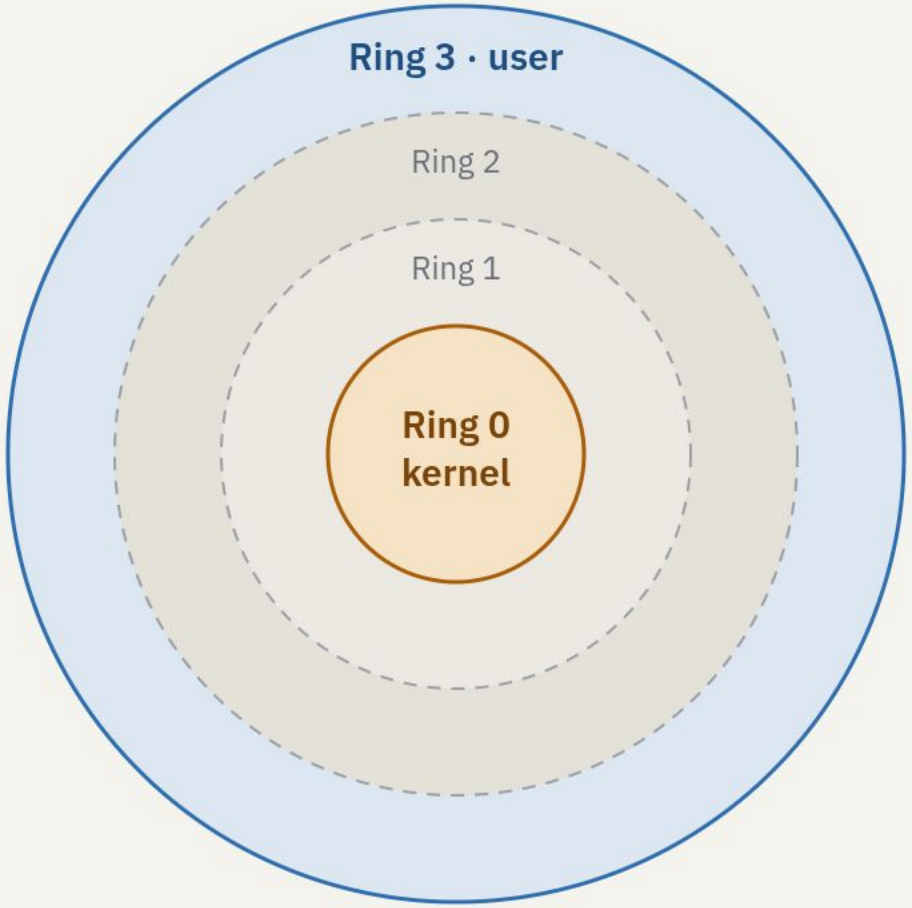
δίχτυ ασφαλείας: προστασία μνήμης

Kernel space

ένα bug → δεν υπάρχει δίχτυ



ring 0 · χωρίς δίχτυ



PRIVILEGE RINGS

Ο επεξεργαστής ξέρει ποιος είναι ο πυρήνας

Η CPU κρατά σε κάθε στιγμή το τρέχον επίπεδο προνομίων (**CPL**· στον x86, τα 2 χαμηλότερα bits του καταχωρητή CS).

Ο x86 ορίζει 4 rings, αλλά το Linux χρησιμοποιεί μόνο δύο: **ring 0** για τον πυρήνα και **ring 3** για τις εφαρμογές.

ARM: EL0 user · EL1 kernel · EL2 hypervisor · EL3 secure monitor

RISC-V: U-mode · S-mode · M-mode

ΤΙ ΕΠΙΒΑΛΛΕΙ ΤΟ HARDWARE

Τρεις φραγμοί για τον κώδικα σε ring 3

Προνομιούχες εντολές

Εντολές που ελέγχουν όλο το μηχάνημα εκτελούνται μόνο σε ring 0.

`cli · hlt · in/out · mov cr3 · wrmsr`

→ από ring 3: **General Protection fault**, ο πυρήνας στέλνει SIGSEGV

Προστασία μνήμης

Κάθε σελίδα έχει bit U/S στον πίνακα σελίδων. Η μνήμη του πυρήνα είναι supervisor-only.

`PTE.U/S = 0 → μόνο ring 0`

→ από ring 3: **page fault**, SIGSEGV

Ελεγχόμενη είσοδος

Δεν μπορείτε να κάνετε jump σε κώδικα του πυρήνα. Μπαίνετε μόνο από «πόρτες» που όρισε ο ίδιος.

`syscall · interrupts · exceptions (IDT)`

→ ο πυρήνας ορίζει **από πού** ξεκινά η εκτέλεση

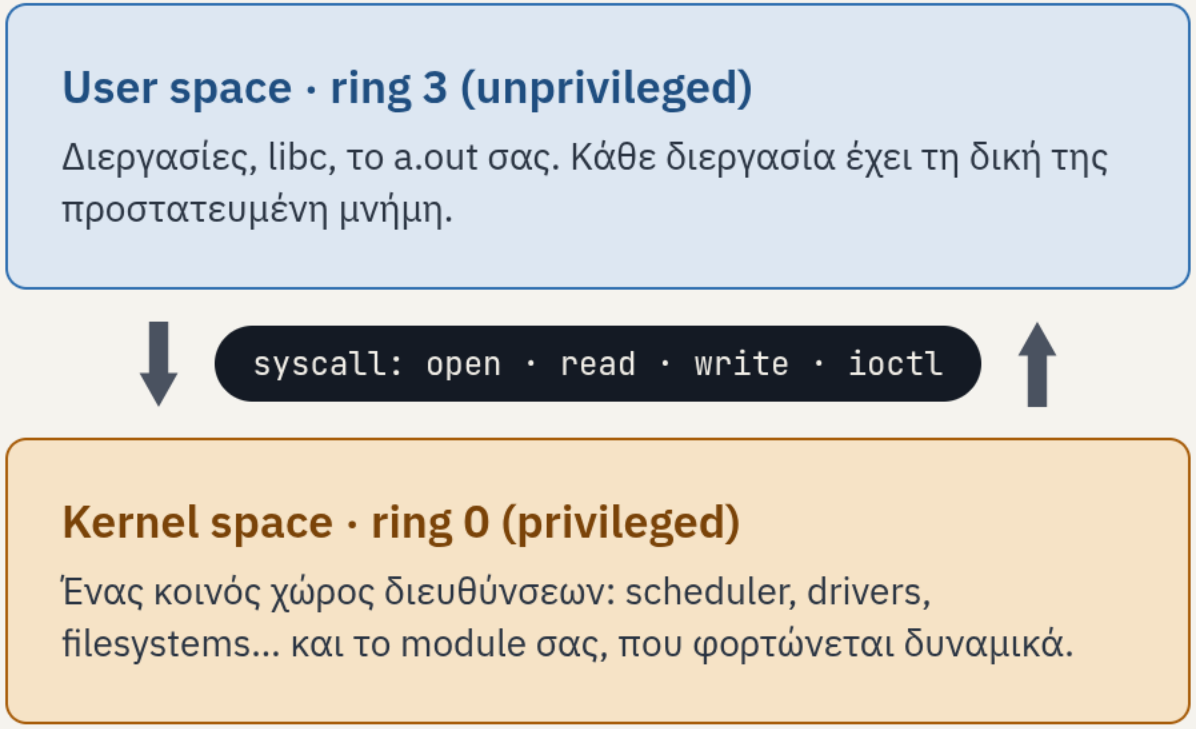
> Ο πυρήνας στήνει αυτούς τους μηχανισμούς στο boot και η CPU τους ελέγχει σε κάθε εντολή. **Σε ring 0 όμως δεν σας προστατεύουν:** γι’ αυτό ένα bug στο module σας μπορεί να επηρεάσει τη λειτουργία του μηχανήματος.

ΑΠΟ ΤΟ USER ΣΤΟ KERNEL SPACE

Αλλάζετε πλευρά

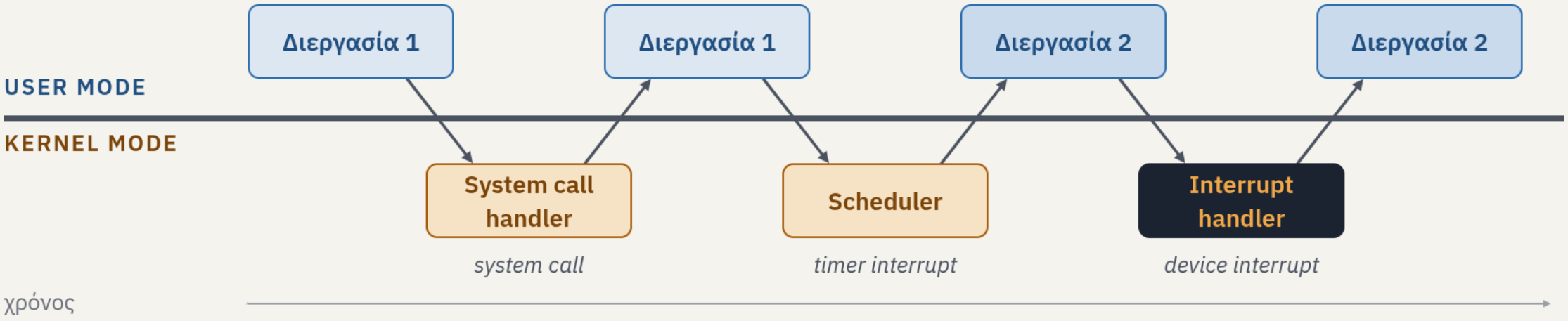
Στο 6ο εξάμηνο ζητούσατε υπηρεσίες από τον πυρήνα μέσω system calls.

Τώρα ο κώδικάς σας τρέχει **μέσα** στον πυρήνα: στον ίδιο χώρο διευθύνσεων με όλο το σύστημα, με πλήρη προνόμια, **χωρίς δίχτυ ασφαλείας**.



KERNEL CONTEXTS

Ο πυρήνας δεν είναι διεργασία



Process context

Εκ μέρους μιας διεργασίας (syscall). Υπάρχει **current**· επιτρέπεται ύπνος και copy_*_user.

Interrupt context

Απόκριση σε hardware event, χωρίς διεργασία. **Δεν κοιμάται**· τελειώνει γρήγορα.

Kernel threads

Διεργασίες μόνο-πυρήνα, χωρίς χώρο χρήστη: [kworker/0:1] στο ps.

PROCESS CONTROL BLOCK

Κάθε διεργασία είναι ένα struct task_struct

Αρχείο include/linux/sched.h:835

```
struct task_struct {
    unsigned int      __state;  /* RUNNING, ... */
    void              *stack;
    ...
    int  prio, static_prio, normal_prio;
    unsigned int      rt_priority;
    ...
    struct mm_struct  *mm, *active_mm;
    ...
    pid_t             pid;
    pid_t             tgid;
    ...
    const struct cred __rcu  *cred;
    char              comm[TASK_COMM_LEN];
    /* open file information */
    struct files_struct  *files;
    ...
};
```

current

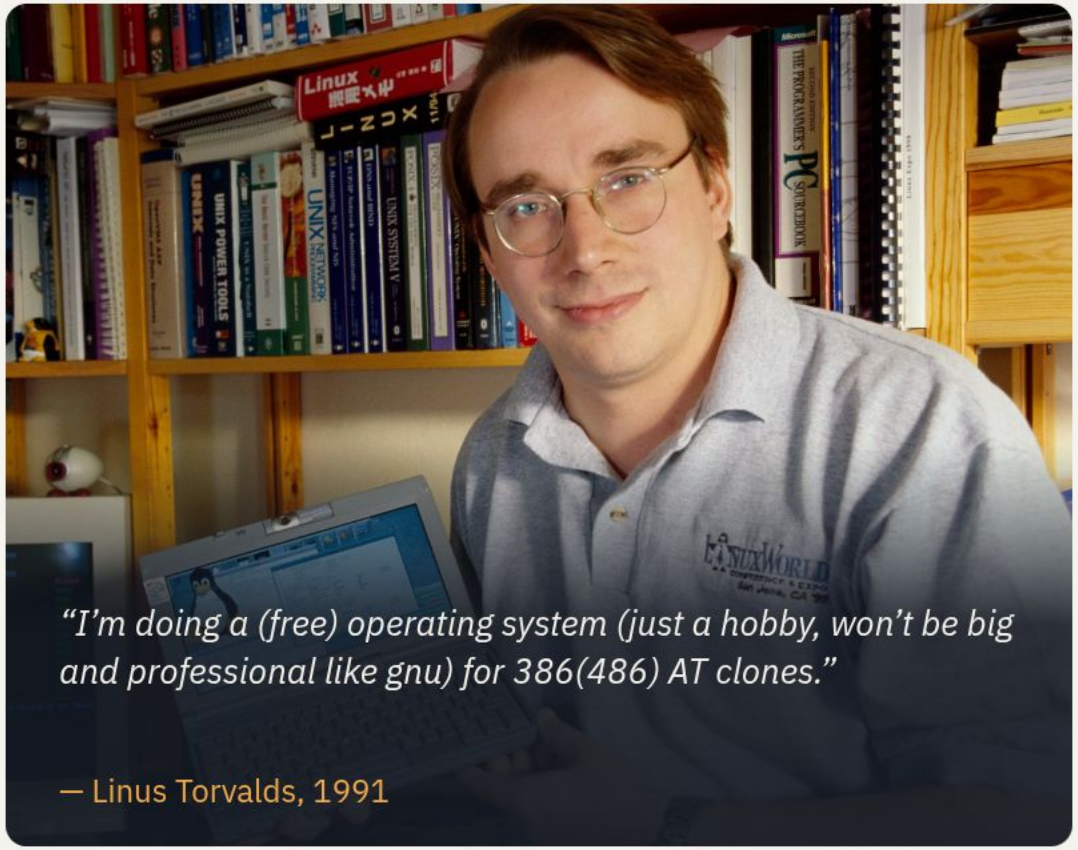
Δείκτης στο task_struct της διεργασίας που τρέχει τώρα (process context).

```
pr_info("%s [%d]\n",
  current→comm,
  current→pid);
```

Threads: κάθε thread έχει δικό του task_struct και pid· όλα μοιράζονται το ίδιο **tgid**, αυτό που επιστρέφει το getpid().

Kernel threads: mm == NULL, δεν έχουν χώρο χρήστη.

Ο πυρήνας του Linux



Linux 7.3 · 18 Οκτωβρίου 2026

96.000+
αρχεία στον πηγαίο κώδικα

43,9 εκατ.
γραμμές

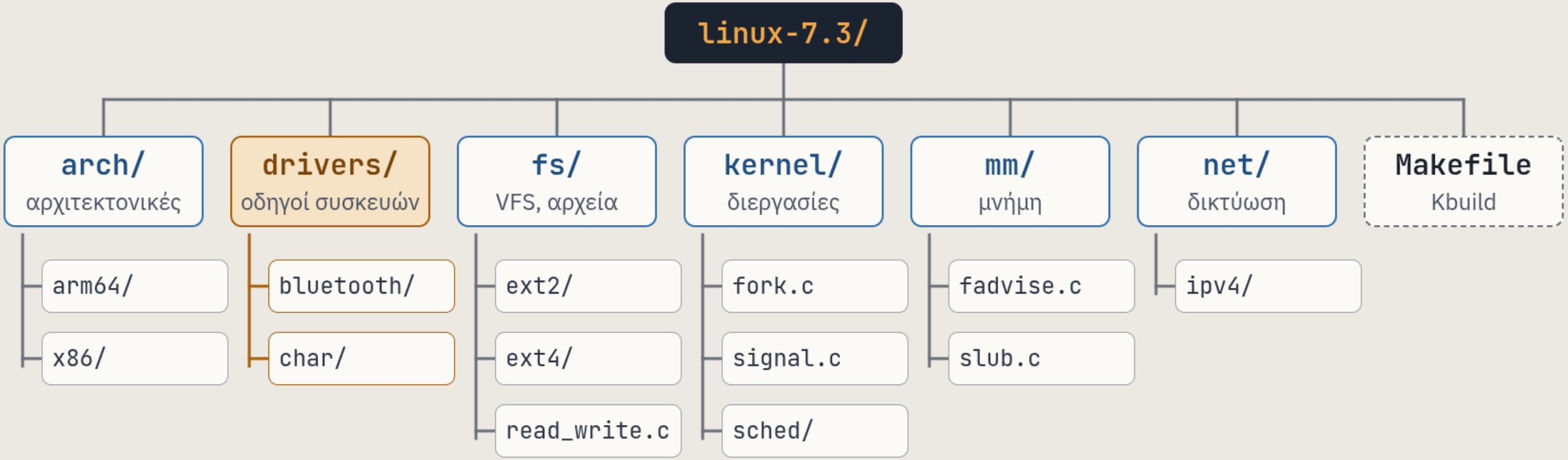
3.021
προγραμματιστές σε αυτή την έκδοση (18.765 commits)

Τρέχει ουσιαστικά παντού: από κινητά και ενσωματωμένα συστήματα μέχρι υπερυπολογιστές.

- `https://kernel.org`
- `https://github.com/torvalds/linux`

ΕΙΣΑΓΩΓΗ

Οργάνωση κώδικα πυρήνα



ΠΛΟΗΓΗΣΗ ΣΤΟΝ ΚΩΔΙΚΑ · 0/4 · ΑΚΟΜΑ ΣΤΟ USER SPACE

Η read() της libc: ένα λεπτό περιτύλιγμα

`read()` → `SYSCALL_DEFINE3(read)` → `ksys_read()` → `vfs_read()` → `f_op→read()`

1 · ΤΟ ΠΡΟΓΡΑΜΜΑ ΣΑΣ

main.c

```c
char buf[100];
ssize_t n;

n = read(fd, buf, 100);
if (n < 0)
    perror("read");
```

→

2 · GLIBC, ΣΕ C

sysdeps/unix/sysv/linux/read.c

```c
ssize_t
__libc_read(int fd, void *buf,
            size_t nbytes)
{
  return SYSCALL_CANCEL(read, fd,
            buf, nbytes);
}
weak_alias(__libc_read, read)
```

→

3 · ΤΙ ΕΚΤΕΛΕΙΤΑΙ (x86-64)

objdump -d libc.so.6

```
read:
  mov  eax, 0  ; __NR_read
  syscall
  cmp  rax, -4095
  jae  .Lerror
  ret
.Lerror: errno = -rax;
  return -1
```

> Η libc **δεν διαβάζει τίποτα**: βάζει τον αριθμό του syscall, εκτελεί την εντολή **syscall** και μετατρέπει το -errno σε -1 / errno. Η πραγματική δουλειά γίνεται στον πυρήνα.

ΠΛΟΗΓΗΣΗ ΣΤΟΝ ΚΩΔΙΚΑ · 1/4

Πού υλοποιείται η read();

`read()` → `SYSCALL_DEFINE3(read)` → `ksys_read()` → `vfs_read()` → `f_op→read()`

USER SPACE · man 2 read

```
ssize_t read(int fd, void *buf, size_t count);
```

⬇

Αρχείο `fs/read_write.c:723`

```
SYSCALL_DEFINE3(read, unsigned int, fd,
        char __user *, buf, size_t, count)
{
    return ksys_read(fd, buf, count);
}
```

SYSCALL_DEFINE3: macro που ορίζει το σημείο εισόδου ενός syscall με 3 ορίσματα, π.χ. __x64_sys_read στο x86-64.

Η διεύθυνσή του μπαίνει στον **sys_call_table**, στη θέση του αριθμού του syscall (read = 0 στο x86-64).

__user: σημειώνει ότι ο buf είναι διεύθυνση του χρήστη· ο πυρήνας δεν τον χρησιμοποιεί απευθείας.

ΠΛΟΗΓΗΣΗ ΣΤΟΝ ΚΩΔΙΚΑ · 2/4

ksys_read(): από το fd στο struct file

read() → SYSCALL_DEFINE3(read) → **ksys_read()** → vfs_read() → f_op→read()

Αρχείο fs/read_write.c:705

```
ssize_t ksys_read(unsigned int fd, char __user *buf, size_t count)
{
    CLASS(fd_pos, f)(fd);
    ssize_t ret = -EBADF;
    if (!fd_empty(f)) {
        loff_t pos, *ppos = file_ppos(fd_file(f));
        if (ppos) {
            pos = *ppos;
            ppos = &pos;
        }
        ret = vfs_read(fd_file(f), buf, count, ppos);
        if (ret ⩾ 0 && ppos)
            fd_file(f)→f_pos = pos;
    }
    return ret;
}
```

CLASS(fd_pos, f)(fd): βρίσκει το struct file που αντιστοιχεί στο fd της διεργασίας. Η αποδέσμευση γίνεται αυτόματα όταν το f βγει εκτός εμβέλειας.

ppos: η τρέχουσα θέση στο αρχείο. Δουλεύουμε σε τοπικό αντίγραφο και την ενημερώνουμε μόνο αν η ανάγνωση πέτυχε.

vfs_read(): η κλήση προς το VFS, το επόμενο βήμα.

vfs_read(): πρώτα οι έλεγχοι

read() → SYSCALL_DEFINE3(read) → ksys_read() → **vfs_read()** → f_op→read()

Αρχείο fs/read_write.c:554

```
ssize_t vfs_read(struct file *file, char __user *buf, size_t count, loff_t *pos)
{
    ssize_t ret;
    if (!(file→f_mode & FMODE_READ))
        return -EBADF;
    if (!(file→f_mode & FMODE_CAN_READ))
        return -EINVAL;
    if (unlikely(!access_ok(buf, count)))
        return -EFAULT;
    ret = rw_verify_area(READ, file, pos, count);
    if (ret)
        return ret;
    if (count > MAX_RW_COUNT)
        count = MAX_RW_COUNT;
    ...
```

f_mode: ανοίχτηκε το αρχείο για ανάγνωση;

access_ok: ο buf βρίσκεται στο εύρος διευθύνσεων του χρήστη;

rw_verify_area: έγκυρη θέση, κλειδώματα αρχείου.

Τα λάθη επιστρέφουν ως **-errno**.

ΠΛΟΗΓΗΣΗ ΣΤΟΝ ΚΩΔΙΚΑ · 4/4

vfs_read(): η κλήση προς τον οδηγό

`read()` → `SYSCALL_DEFINE3(read)` → `ksys_read()` → `vfs_read()` → **`f_op→read()`**

Αρχείο `fs/read_write.c:554`

```
ssize_t vfs_read(struct file *file, char __user *buf, size_t count, loff_t *pos)
{
    ...
    if (file→f_op→read)
        ret = file→f_op→read(file, buf, count, pos);
    else if (file→f_op→read_iter)
        ret = new_sync_read(file, buf, count, pos);
    else
        ret = -EINVAL;
    if (ret > 0) {
        fsnotify_access(file);
        add_rchar(current, ret);
    }
    inc_syscr(current);
    return ret;
}
```

Εδώ μπαίνει ο driver σας

Το f_op είναι το struct file_operations του αρχείου. Για ένα character device που αφορά τους αισθητήρες , το .read είναι η δική σας συνάρτηση.

current: δείκτης στη διεργασία που έκανε την κλήση (process context).

ΣΥΝΟΨΗ · ΕΚΤΕΛΩΝΤΑΣ ΕΝΑ SYSTEM CALL

Τι συμβαίνει σε ένα write() στο x86-64

RING 3

libc

Αριθμός syscall στον rax, ορίσματα σε rdi, rsi, rdx.

→

3 → 0

`syscall`

Η CPU γίνεται CPL 0 και πηδά στο entry point που όρισε ο πυρήνας.

→

RING 0

Είσοδος

Kernel stack, αποθήκευση καταχωρητών, sys_call_table[rax].

→

RING 0

Ο driver σας

VFS → η .write του module σας.

→

0 → 3

`sysret`

Πίσω σε CPL 3· η τιμή επιστροφής στον rax.

> Ο κώδικας του module σας τρέχει σε ring 0, αλλά **για λογαριασμό** της διεργασίας που έκανε την κλήση (process context). Ο **buf** που σας δίνει είναι διεύθυνση user space: γι’ αυτό copy_from_user().

Το write() της libc σε assembly

x86-64

```
write:
; rdi=fd rsi=buf rdx=count
  mov  eax, 1      ; __NR_write
  syscall          ; → ring 0
  cmp  rax, -4095
  jae  .Lerror     ; -errno
  ret
```

Αριθμός στον **rax**, αποτέλεσμα στον **rax**.

arm64 (AArch64)

```
write:
// x0=fd x1=buf x2=count
  mov  x8, #64     // __NR_write
  svc  #0          // → EL1
  cmn  x0, #4095
  b.cs .Lerror     // -errno
  ret
```

Αριθμός στον **x8**, αποτέλεσμα στον **x0**.

> Τα ορίσματα της C είναι **ήδη** στους σωστούς καταχωρητές: η libc βάζει μόνο τον αριθμό του syscall και εκτελεί μία εντολή. Αν πάρει πίσω τιμή από −4095 ως −1, ορίζει το errno και επιστρέφει −1.

ΠΛΟΗΓΗΣΗ ΣΤΟΝ ΚΩΔΙΚΑ · ΕΡΓΑΛΕΙΟ

Elixir Cross Referencer

`elixir.bootlin.com/linux/v7.3/source`

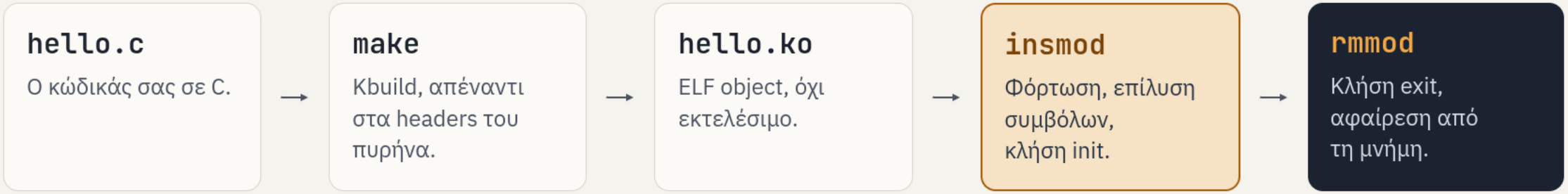
Online browser του κώδικα του πυρήνα: κάθε σύμβολο είναι σύνδεσμος στον ορισμό του και σε όλες τις χρήσεις του.

- Εύκολη αναζήτηση ορισμών και αναφορών
- Κώδικας από όλες τις εκδόσεις του πυρήνα
- Και άλλα projects εκτός από τον πυρήνα (π.χ. QEMU)

KERNEL MODULES

Κομμάτια πυρήνα που φορτώνονται εν λειτουργία

Ένα module είναι object code (**.ko**) που μπαίνει στον πυρήνα ενώ αυτός τρέχει, χωρίς reboot ή recompile. Μόλις φορτωθεί, **είναι** πυρήνας: ring 0, ίδιος χώρος διευθύνσεων, και καλεί μόνο ό,τι ο πυρήνας εξάγει (EXPORT_SYMBOL).



Built-in (=y)

Μέσα στο vmlinux από το boot· δεν αφαιρείται.

Module (=m)

Στο /lib/modules/$(uname -r)/· φορτώνεται όταν χρειαστεί.

KERNEL MODULES · BUILD & DEBUG

Kbuild αντί για gcc, dmesg αντί για gdb

Το module χτίζεται με το build system του ίδιου του πυρήνα, απέναντι στα headers της έκδοσης που τρέχει.

```
obj-m := hello.o
KDIR := /lib/modules/$(shell uname -r)/build

all:
    $(MAKE) -C $(KDIR) M=$(PWD) modules
clean:
    $(MAKE) -C $(KDIR) M=$(PWD) clean
```

Δεν υπάρχει σταθερό εσωτερικό API: κώδικας για μία έκδοση μπορεί να μη χτίζεται σε άλλη.

Εντολή	Τι κάνει
sudo insmod hello.ko	φορτώνει, καλεί init
sudo rmmod hello	αφαιρεί, καλεί exit
lsmod	φορτωμένα modules
modinfo hello.ko	άδεια, παράμετροι
dmesg -w	ζωντανά τα printk

Debugging: κυρίως printk και dmesg. Για τα δύσκολα, QEMU + gdb πάνω στον πυρήνα του VM.

Δεν υπάρχει main()

Πρόγραμμα χρήστη

```c
#include <stdio.h>

int main(void)
{
    printf("Hello\n");
    return 0;
}
```

```
$ gcc hello.c && ./a.out
```

Kernel module

```c
#include <linux/module.h>

static int __init hello_init(void)
{
    pr_info("Hello, kernel\n");
    return 0;
}
static void __exit hello_exit(void)
{
    pr_info("Bye\n");
}
module_init(hello_init);
module_exit(hello_exit);
MODULE_LICENSE("GPL");
```

```
$ make && sudo insmod hello.ko && dmesg
$ rmmod hello
```

Η **init** τρέχει στο insmod και η **exit** στο rmmod.

Καμία libc: μόνο ό,τι δίνει ο πυρήνας

Θέμα	User space	Kernel space
Έξοδος	printf()	printk() / pr_info() → διαβάζετε με dmesg
Μνήμη	malloc() / free()	kmalloc(size, GFP_KERNEL) / kfree(), vmalloc()
Headers	<stdio.h>, <stdlib.h>	μόνο <linux/...>, <asm/...>
Λάθη	return -1 και errno	return -ENOMEM, -EINVAL, -EFAULT...
Δείκτες χρήστη	απευθείας χρήση	copy_from_user() / copy_to_user(), ποτέ απευθείας
Στοίβα	μερικά MB, μεγαλώνει	~8–16 KB, σταθερή: όχι μεγάλα τοπικά arrays
Floating point	ελεύθερα	όχι (εκτός ειδικών περιπτώσεων)

ΛΑΘΗ

Ένα bug ρίχνει όλο το σύστημα

User space: segfault → πεθαίνει μόνο η διεργασία σας.

Kernel: NULL dereference → Oops ή kernel panic. Μπορεί να κρεμάσει το μηχάνημα ή να χαλάσει δεδομένα σιωπηλά.

- Δουλεύετε σε εικονική μηχανή (VM)
- Ελέγχετε κάθε τιμή επιστροφής
- Αποδεσμεύετε σε αντίστροφη σειρά: το μοτίβο goto

Cleanup με goto: το ιδίωμα του πυρήνα

```
buf = kmalloc(SZ, GFP_KERNEL);
if (!buf)
    return -ENOMEM;
err = register_chrdev_region(dev, 1, "mydev");
if (err)
    goto out_free;
err = cdev_add(&mycdev, dev, 1);
if (err)
    goto out_unreg;
return 0;

out_unreg:
    unregister_chrdev_region(dev, 1);
out_free:
    kfree(buf);
    return err;
```

Η init καταχωρεί, η exit αναιρεί

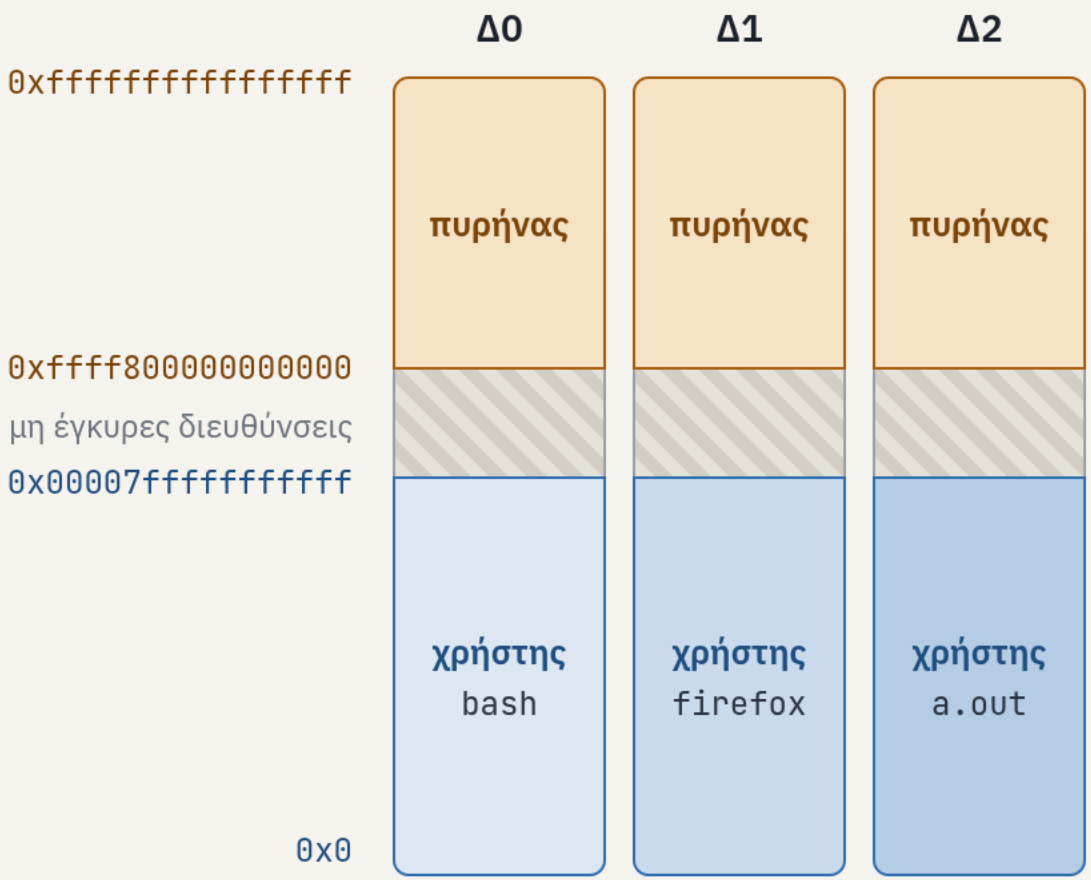
Το module δεν δουλεύει μόνο του. Η init λέει σε ένα υποσύστημα «εδώ είναι οι συναρτήσεις μου» και επιστρέφει. Η exit αναιρεί ακριβώς ό,τι έκανε η init.

1. **Headers** του πυρήνα
2. **Μεταδεδομένα**: MODULE_LICENSE, AUTHOR…
3. **Παράμετροι** φόρτωσης: module_param
4. **init / exit** και καταχώρηση

```
/* 1 */
#include <linux/module.h>
#include <linux/moduleparam.h>
/* 2 */
MODULE_LICENSE("GPL");
MODULE_AUTHOR("OSLAB");
/* 3 */
static int count = 1;
module_param(count, int, 0444);
/* 4 */
static int __init demo_init(void)
{
    pr_info("demo: count=%d\n", count);
    /* καταχώρηση, π.χ. cdev_add() */
    return 0;
}
static void __exit demo_exit(void)
{
    /* αναίρεση, π.χ. cdev_del() */
}
module_init(demo_init);
module_exit(demo_exit);
```

```
$ sudo insmod demo.ko count=5
```

Ένας χώρος διευθύνσεων, δύο κόσμοι



Χώρος χρήστη

Διαφορετικός για κάθε διεργασία. Η διεργασία βλέπει μόνο αυτόν.

Χώρος πυρήνα

Ίδιος σε όλες τις διεργασίες, προσβάσιμος μόνο σε ring 0. Από εδώ ο πυρήνας βλέπει τα πάντα.

ΑΝΑΜΕΣΑ ΣΤΟΥΣ ΔΥΟ

```
copy_from_user(): χρήστης → πυρήνας
copy_to_user():   πυρήνας → χρήστης
```

ΔΙΑΧΕΙΡΙΣΗ ΜΝΗΜΗΣ · 2/3

Πώς δεσμεύουμε μνήμη

USER SPACE

`malloc() / free()`

↓

allocator της libc

↓

`brk() / mmap()` `syscall`

↓

Οι σελίδες δίνονται **όταν τις αγγίξετε** (page fault)

KERNEL

`kmalloc() / kfree()`
μικρά, **φυσικά** συνεχόμενα

`vmalloc() / vfree()`
μεγάλα, **εικονικά** συνεχόμενα

↓

SLUB caches
`kmalloc-64, task_struct…`

πίνακες σελίδων
σκόρπιες σελίδες → ένα κομμάτι

↓

Page allocator `alloc_pages()` · `__get_free_pages()`
μονάδα: η σελίδα (struct page) · ζώνες DMA, DMA32, Normal

`GFP_KERNEL` μπορεί να κοιμηθεί · `GFP_ATOMIC` ποτέ (interrupts, spinlocks)

Τι σημαίνει αυτό για τον κώδικά σας

	User space	Kernel
Αποτυχία δέσμευσης	σπάνια στην πράξη	πάντα έλεγχος για NULL → -ENOMEM
Swap / page faults	φυσιολογικά, αόρατα	η μνήμη του πυρήνα δεν γίνεται swap
Leak	καθαρίζει στο exit()	μένει μέχρι το reboot
Λάθος δείκτης	SIGSEGV	Oops, panic ή σιωπηλή αλλοίωση
Μεγάλα buffers	malloc(1 GB): ok	kmalloc για μικρά, vmalloc για μεγάλα
Εργαλεία	valgrind, ASan	KASAN, kmemleak, /proc/slabinfo

> Κάθε kmalloc στην init έχει το kfree του στην exit, και κάθε μονοπάτι λάθους αποδεσμεύει ό,τι δέσμευσε.

ΤΑΥΤΟΧΡΟΝΙΣΜΟΣ

Όλα τρέχουν ταυτόχρονα

Πολλές διεργασίες

Πολλοί ανοίγουν και διαβάζουν τη συσκευή σας ταυτόχρονα, σε διαφορετικούς πυρήνες CPU.

Interrupts

Ένας interrupt handler μπορεί να διακόψει τον κώδικά σας ανάμεσα σε δύο εντολές.

Preemption

Ο scheduler μπορεί να σας αφαιρέσει τη CPU σχεδόν οποτεδήποτε.

`spinlock_t`

Σύντομα κρίσιμα τμήματα· χρησιμοποιείται και σε interrupt context (spin_lock_irqsave).

`struct mutex / semaphore`

Μπορεί να κοιμηθεί (block)· μόνο σε process context.

> **Κανόνας:** όσο κρατάτε spinlock ή είστε σε interrupt, δεν κοιμάστε. Άρα όχι mutex, όχι kmalloc(GFP_KERNEL), όχι copy_from_user().

ΤΑΥΤΟΧΡΟΝΙΣΜΟΣ

Μηχανισμοί συγχρονισμού: τα API

Ατομικές πράξεις — `atomic_t`

```
atomic_t cnt = ATOMIC_INIT(0);
atomic_inc(&cnt);
n = atomic_read(&cnt);
```

Ένας ακέραιος, χωρίς κλείδωμα.

Spinlocks — `spinlock_t`

```
spin_lock(&lock);
spin_lock_irqsave(&lock, flags);
spin_unlock_irqrestore(&lock, flags);
```

Σύντομα τμήματα· και σε interrupt context.

Σημαφόροι — `struct semaphore`

```
sema_init(&sem, 1);
if (down_interruptible(&sem))
    return -ERESTARTSYS;
up(&sem);
```

Μπορεί να κοιμηθεί· μόνο σε process context.

Wait queues — `wait_queue_head_t`

```
if (wait_event_interruptible(wq,
        has_new_data()))
    return -ERESTARTSYS;
wake_up_interruptible(&wq);
```

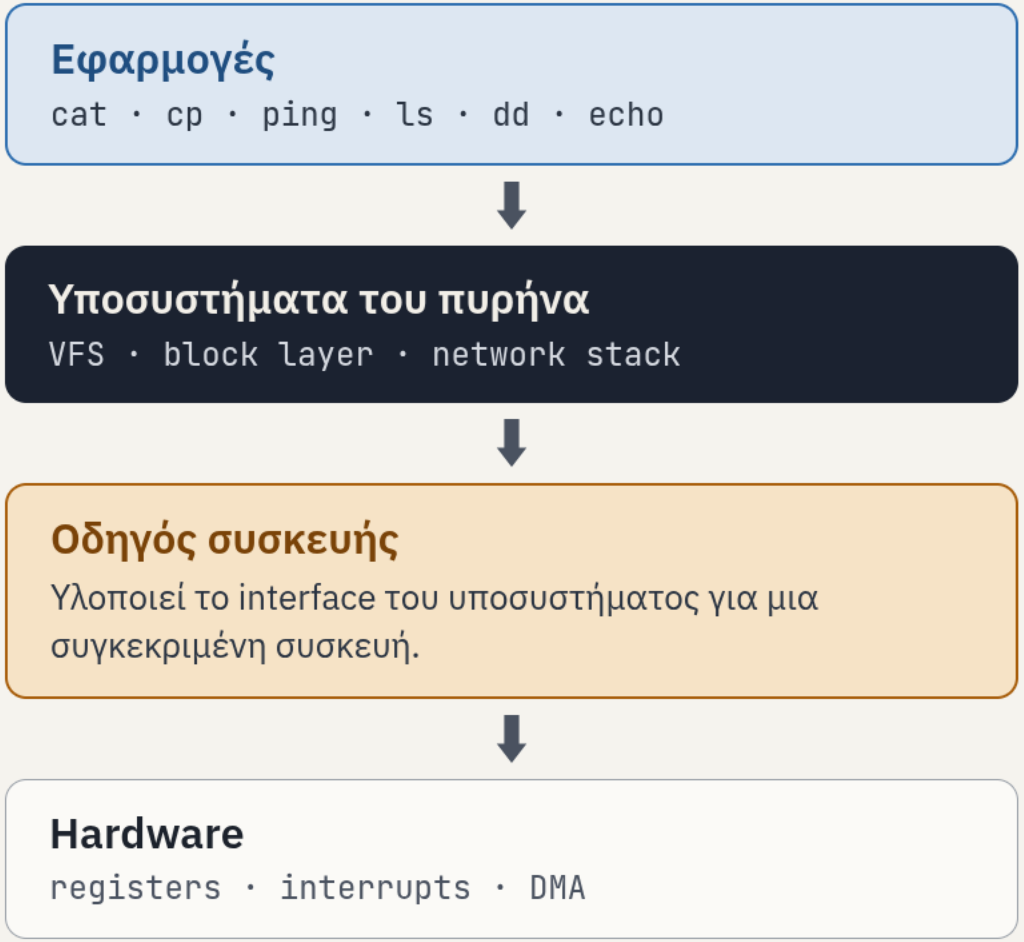
Ύπνος μέχρι να ισχύσει μια συνθήκη.

DEVICE DRIVERS

Ο οδηγός κρύβει το hardware πίσω από ένα τυπικό interface

Οι εφαρμογές δεν ξέρουν αν ο δίσκος είναι SATA ή NVMe. Ο οδηγός μιλά τη «γλώσσα» της συσκευής προς τα κάτω και τη γλώσσα του πυρήνα προς τα πάνω.

Ο μεγαλύτερος κατάλογος στον πηγαίο κώδικα του Linux είναι το **drivers/**, και οι περισσότεροι οδηγοί χτίζονται ως modules.



ΤΡΙΑ ΕΙΔΗ ΟΔΗΓΩΝ ΣΥΣΚΕΥΩΝ ΣΤΟ LINUX

Character, block, network

Character

Ροή bytes, συνήθως σειριακή πρόσβαση.

`/dev/ttyS0 · /dev/input/*`

`file_operations`

Block

Blocks σταθερού μεγέθους, τυχαία πρόσβαση, μέσα από page cache και I/O scheduler.

`/dev/sda · /dev/nvme0n1`

`block_device_operations · blk-mq`

Network

Πακέτα, όχι bytes. Κανένα αρχείο στο /dev: εμφανίζονται ως interfaces.

`eth0 · wlan0 (ip link)`

`net_device_ops · sk_buff`

```
$ ls -l /dev/ttyS0 /dev/sda
crw-rw---- 1 root dialout 4, 64 /dev/ttyS0
brw-rw---- 1 root disk    8,  0 /dev/sda

$ ls -l /var/run/snapd.socket
srw-rw-rw- 1 root root 0 Oct 20 11:51 /var/run/snapd.socket
```

c: character
b: block
s: socket
4, 64: major (ποιος οδηγός), minor (ποια συσκευή του).

TO DEVICE MODEL

Bus, device, driver και probe()

1. Το **bus** (PCI, USB, platform / device tree) ανακαλύπτει μια συσκευή και τα IDs της.
2. Κάθε **driver** δηλώνει πίνακα με τα IDs που υποστηρίζει.
3. Ο πυρήνας κάνει **match** και καλεί την probe(). Όταν η συσκευή φύγει, την remove().
4. Το **udev** φορτώνει αυτόματα το σωστό module και φτιάχνει το αρχείο στο /dev.

Μέσα στην probe()

ioremap() για τους registers, request_irq() για τα interrupts, καταχώρηση στο υποσύστημα.

```
static const struct pci_device_id my_ids[] = {
    { PCI_DEVICE(0x8086, 0x100e) }, /* e1000 */
    { }
};
MODULE_DEVICE_TABLE(pci, my_ids);

static struct pci_driver my_driver = {
    .name     = "mydrv",
    .id_table = my_ids,
    .probe    = my_probe,
    .remove   = my_remove,
};
module_pci_driver(my_driver);
```

```
$ lspci -k    # ποιος driver ανά συσκευή
$ ls /sys/bus/pci/drivers/
```

ΥΛΟΠΟΙΗΣΗ: CHARACTER DEVICE

Από το /dev/mydev στις συναρτήσεις σας

RING 3

```
$ cat /dev/mydev
```

⬇

```
open() · read() → syscall
```

⬇

VFS: ο αριθμός major/minor του αρχείου βρίσκει το cdev σας

⬇

RING 0 · ΤΟ MODULE ΣΑΣ

```
fops.read → mydev_read()
```

1. Ο πίνακας λειτουργιών

```
static const struct file_operations
    mydev_fops = {
    .owner   = THIS_MODULE,
    .open    = mydev_open,
    .read    = mydev_read,
    .release = mydev_release,
};
```

2. Καταχώρηση στην init

```
alloc_chrdev_region(&dev, 0, 1, "mydev");
cdev_init(&mycdev, &mydev_fops);
cdev_add(&mycdev, dev, 1);
/* + έλεγχος λαθών με goto */
```

ΥΛΟΠΟΙΗΣΗ: READ

Μια read που τηρεί τους κανόνες

```
static ssize_t mydev_read(struct file *filp,
        char __user *ubuf, size_t len,
        loff_t *off)
{
    static const char msg[] = "hello\n";
    size_t avail = sizeof(msg) - 1;

    if (*off ⩾ avail)
        return 0;        /* EOF */
    len = min_t(size_t, len, avail - *off);
    if (copy_to_user(ubuf, msg + *off, len))
        return -EFAULT;
    *off += len;
    return len;
}
```

__user: ο δείκτης ανήκει στον χρήστη. Ποτέ απευθείας· μόνο copy_to_user().

***off:** η θέση στο «αρχείο». Το return 0 σημαίνει EOF, αλλιώς το cat δεν σταματά.

Τιμή επιστροφής: πόσα bytes αντιγράψατε, ή -errno.

```
$ sudo insmod mydev.ko
$ grep mydev /proc/devices
$ sudo mknod /dev/mydev c MAJOR 0
$ cat /dev/mydev
hello
```

Πέντε κανόνες επιβίωσης στον Πυρήνα

01 Δουλεύετε σε VM, όχι στο λάπτοπ σας.

02 Ελέγχετε κάθε τιμή επιστροφής και καθαρίζετε ό,τι δεσμεύσατε.

03 Ποτέ δεν εμπιστεύεστε δείκτη που ήρθε από το user space.

04 Πάντα αναρωτιέστε: ποιος άλλος τρέχει αυτόν τον κώδικα αυτή τη στιγμή;

05 Διαβάζετε τον κώδικα του πυρήνα: είναι η καλύτερη τεκμηρίωση (elixir.bootlin.com).

ΓΙΑ ΠΕΡΑΙΤΕΡΩ ΜΕΛΕΤΗ

Βιβλιογραφία

ΤΑ ΚΛΑΣΙΚΑ

Linux Kernel Development
Robert Love · 3η έκδ., Addison-Wesley, 2010

Understanding the Linux Kernel
D. Bovet, M. Cesati · 3η έκδ., O'Reilly, 2006

Linux Device Drivers
J. Corbet, A. Rubini, G. Kroah-Hartman · 3η έκδ., O'Reilly, 2005
`lwn.net/Kernel/LDD3/ (δωρεάν)`

ΠΙΟ ΣΥΓΧΡΟΝΑ

Linux Kernel Programming
Kaiwan N. Billimoria · 2η έκδ., Packt, 2024 · kernel 6.1

The Linux Kernel Module Programming Guide
Online, ενημερωμένος για kernel 5.x / 6.x
`sysprog21.github.io/lkmpg`

Η τεκμηρίωση του ίδιου του πυρήνα
`docs.kernel.org`

Ευχαριστούμε!

Επικοινωνία με τους βοηθούς του μαθήματος
`cslab-assist@cslab.ece.ntua.gr`